

Project Title: Smart Energy Consumption Analysis and Prediction using Machine Learning with Device-Level Insights

Milestone 1: Week 1-2

-by Partha Mitra

Objective: Data Collection and Understanding

It is the first phase of my project, where I imported all the necessary libraries to study the dataset.

Dataset used is a large csv file of having Columns:

Home ID	Appliance Type	Energy Consumption (kWh)	Time	Date	Outdoor Temperature (°C)	Season	Household Size
---------	----------------	--------------------------	------	------	--------------------------	--------	----------------

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Home ID                               100000 non-null int64
1   Appliance Type                        100000 non-null object
2   Energy Consumption (kWh)              100000 non-null float64
3   Time                                  100000 non-null object
4   Date                                  100000 non-null object
5   Outdoor Temperature (°C)             100000 non-null float64
6   Season                                100000 non-null object
7   Household Size                        100000 non-null int64
```

Total number of entries is 1,00,000

Checking Null Values:

Checked if there any null values

Outlier Removal:

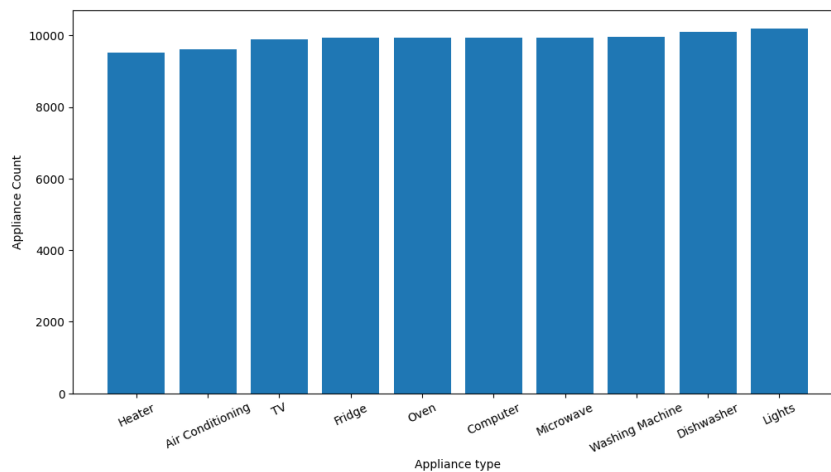
I have removed the outlier, by taking the 99%ile of data according to the “Energy Consumption (kWh)”

Different Devices:

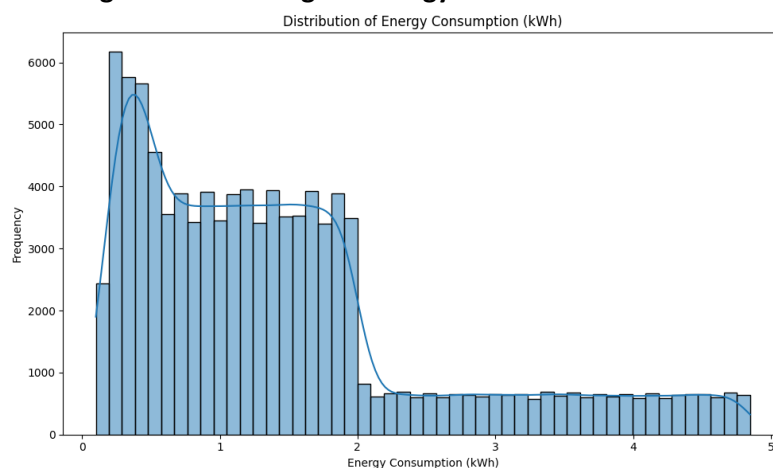
After outlier removal the number of devices we get:

Appliance Type

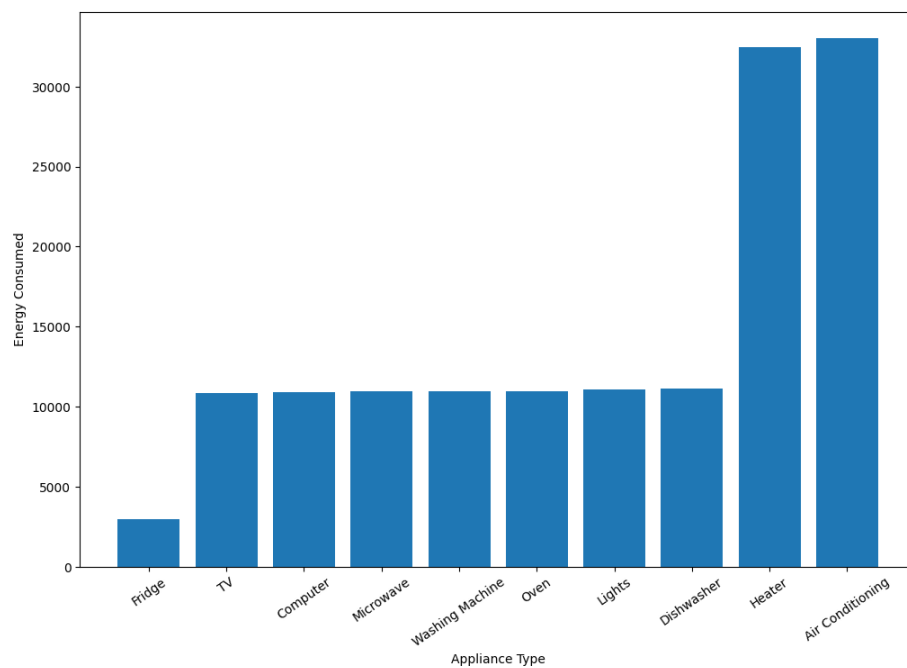
- | | | |
|-------|------------------|-------|
| i. | Heater | 9515 |
| ii. | Air Conditioning | 9619 |
| iii. | TV | 9905 |
| iv. | Fridge | 9931 |
| v. | Oven | 9939 |
| vi. | Computer | 9944 |
| vii. | Microwave | 9947 |
| viii. | Washing Machine | 9953 |
| ix. | Dishwasher | 10095 |
| x. | Lights | 10201 |
| xi. | | |



Checking what is the range of Energy that most devices used:



Total Consumption per Device:



Air-conditioning Contributes to most in Energy consumptions and Fridge the Least.

Timestamp Column

I created another column, 'timestamp' merging Date and Time, for efficiently working with time.

And then dropped the Date and Time Column, since we don't need them anymore

	Home ID	Appliance Type	Energy Consumption (kWh)	Outdoor Temperature (°C)	Season	Household Size	timestamp
0	94	Fridge	0.20	-1.0	Fall	2	2023-12-02 21:12:00
1	435	Oven	0.23	31.1	Summer	5	2023-08-06 20:11:00
2	466	Dishwasher	0.32	21.3	Fall	3	2023-11-21 06:39:00

Objective: Feature Engineering

In this phase, I Extracted the 'hour', 'day', 'month', 'week' from the 'timestamp' column

- Sorted them according to **Appliance Type, timestamp**

Then added **lag features** are created from the **Energy Consumption (kWh)** column to capture temporal dependency in time-series data.

shift(1) creates a **1-step lag feature**, where each value represents the energy consumption of the **previous time step** (e.g., previous hour or previous record).

shift(24) creates a **24-step lag feature**, which represents the energy consumption from **24 time steps earlier**, commonly used to model **daily patterns** in hourly energy data.

Rolling_mean_24 to calculate the mean of previous 24 records

These lag features help the model learn historical trends and repeating patterns in energy consumption. They are particularly useful in **time-series forecasting**

	Home ID	Appliance Type	Energy Consumption (kWh)	Outdoor Temperature (°C)	Season	Household Size	timestamp	hour	day	month	week	lag_1	lag_24	rolling_mean_24	
	62309	498	Air Conditioning	3.91	5.7	Winter	1	2023-01-01 18:40:00	18	1	1	6	2.81	4.42	3.510417
	57699	189	Air Conditioning	3.66	12.9	Winter	3	2023-01-01 18:43:00	18	1	1	6	3.91	4.37	3.480833
	67394	74	Air Conditioning	2.74	6.6	Winter	4	2023-01-01 19:31:00	19	1	1	6	3.66	3.24	3.460000
	82363	266	Air Conditioning	3.49	2.1	Winter	3	2023-01-01 21:50:00	21	1	1	6	2.74	3.75	3.449167
	26057	300	Air Conditioning	4.76	-3.8	Winter	4	2023-01-01 23:56:00	23	1	1	6	3.49	4.22	3.471667
	517	133	Air Conditioning	3.51	6.8	Winter	3	2023-01-02 00:49:00	0	2	1	0	4.76	2.49	3.514167
	94104	187	Air Conditioning	4.41	36.5	Winter	3	2023-01-02 01:40:00	1	2	1	0	3.51	4.34	3.517083
	97269	455	Air Conditioning	2.11	4.9	Winter	2	2023-01-02 02:28:00	2	2	1	0	4.41	4.19	3.430417
	24067	455	Air Conditioning	3.47	13.1	Winter	5	2023-01-02 03:36:00	3	2	1	0	2.11	4.00	3.408333
	94122	71	Air Conditioning	4.74	-10.0	Winter	5	2023-01-02 04:10:00	4	2	1	0	3.47	4.48	3.419167
	43428	59	Air Conditioning	4.37	19.3	Winter	4	2023-01-02 04:16:00	4	2	1	0	4.74	2.20	3.509583
	28121	118	Air Conditioning	4.19	19.3	Winter	5	2023-01-02 06:24:00	6	2	1	0	4.37	4.12	3.512500

Then checked if there any null values, if there are any null values , I dropped them

First, I **iterate through each appliance type** using a loop so that the data can be processed separately for every appliance.

Then, I **filter the dataset** to keep only the records corresponding to the current appliance and create a copy of this filtered data to avoid changes to the original dataset.

Next, I **separate the input features and the target variable**. I remove columns such as *Energy Consumption (kWh)*, *Appliance Type*, *Season*, *Home ID*, and *timestamp* from the dataset to form the feature set, while *Energy Consumption (kWh)* is selected as the target variable.

After that, I **split the data into training and testing sets**. I use 85% of the data for training the model and the remaining 15% for testing, ensuring that the split is done sequentially to maintain the temporal order of the data.

Finally, I **save the training and testing datasets** for each appliance separately as CSV files. These files are stored in a designated folder and are later used for appliance-wise model training and evaluation.

```
x_train.head()
```

✓ 0.0s

	Outdoor Temperature (°C)	Household Size	hour	day	month	week	lag_1	lag_24	rolling_mean_24
92541	2.7	3	0	1	1	6	0.95	0.90	1.150417
38068	5.0	2	1	1	1	6	1.12	1.51	1.170000
91598	-1.6	4	2	1	1	6	1.98	0.21	1.230833
51529	38.9	4	2	1	1	6	1.67	0.63	1.243333
21658	12.6	3	5	1	1	6	0.93	1.11	1.240000

```
y_train.head()
```

✓ 0.0s

92541	1.12
38068	1.98
91598	1.67
51529	0.93
21658	1.03

Name: Energy Consumption (kWh), dtype: float64

```
x_test.head()
```

✓ 0.0s

	Outdoor Temperature (°C)	Household Size	hour	day	month	week	lag_1	lag_24	rolling_mean_24
59815	14.5	5	18	4	11	5	0.76	1.95	1.366250
47134	21.6	3	19	4	11	5	1.80	1.94	1.299167
68612	-8.0	3	19	4	11	5	0.33	1.98	1.270833
55434	0.7	2	21	4	11	5	1.30	1.91	1.259583
57979	14.0	5	21	4	11	5	1.64	0.91	1.262500

```
y_test.head()
```

✓ 0.0s

59815	1.80
47134	0.33
68612	1.30
55434	1.64
57979	0.98

Name: Energy Consumption (kWh), dtype: float64